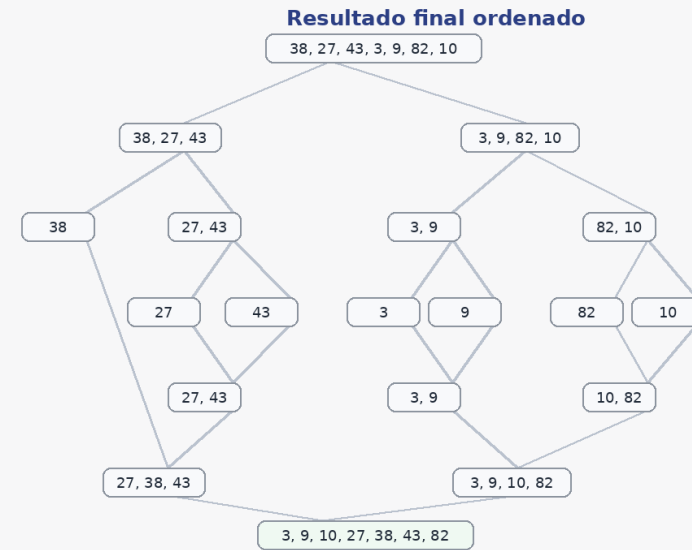


Diseño de Datos y Algoritmos

Dividir y Conquistar

Merge Sort · Teorema Maestro · Búsqueda · Subarreglo
máximo

2026-2



Resolver problemas grandes construyendo
respuestas desde subproblemas.

Agenda de trabajo

Del patrón divide y conquista a búsquedas eficientes y subarreglo máximo.

- | | | |
|-----------|--------------------------|--|
| 01 | Merge Sort | Especificación, diseño, código y análisis. |
| 02 | Teorema Maestro | Resolución rápida de recurrencias típicas. |
| 03 | Búsqueda | Secuencial vs binaria sobre datos ordenados. |
| 04 | Subarreglo máximo | Modelar ganancia como suma máxima. |
| 05 | Ejercicios | Práctica de diseño y complejidad. |

03

BLOQUE 1

Merge Sort

El ejemplo central para entender dividir, conquistar y combinar.

Especificación

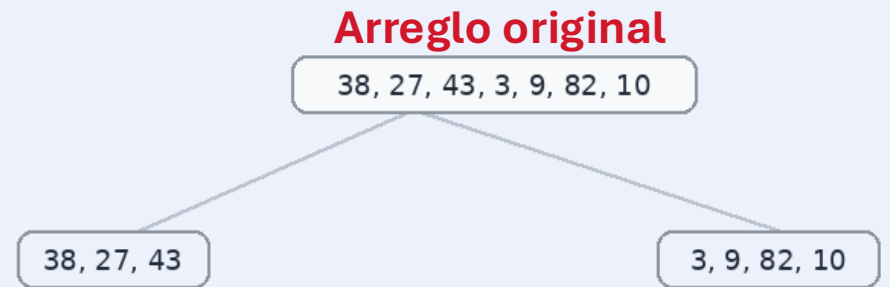
Entrada, salida y condición de corrección.

Entrada Una secuencia de n números.

Salida Una permutación ordenada de los mismos elementos.

Algoritmo correcto

Para toda entrada válida debe producir una salida ordenada que conserve exactamente los elementos originales.



Idea clave

La especificación define qué resolver.
Antes de diseñar, hay que precisar la corrección.

Paradigma

Dividir, conquistar y combinar.



Idea clave

Dividir reduce el problema.

Conquistar resuelve cada subproblema.

Combinar arma la respuesta final.

Merge Sort

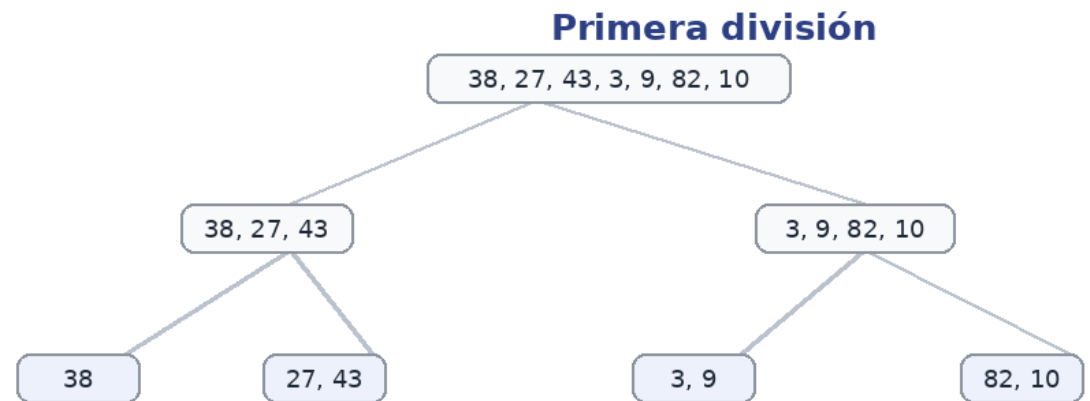
Aplicación directa de divide y conquista.

Dividir el arreglo en mitades.

Ordenar cada mitad recursivamente.

Mezclar subarreglos ordenados.

El caso base ocurre cuando el tamaño es 1.



Merge Sort

Construcción visual paso a paso.

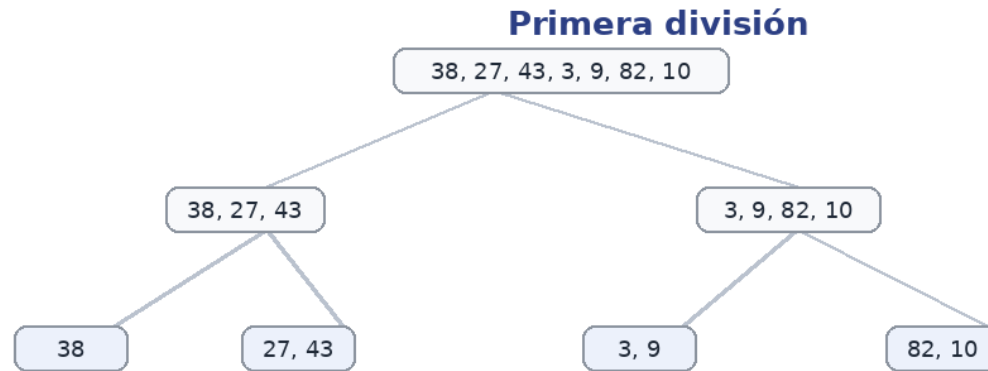


Paso 1

Partimos del problema completo y aún no hay descomposición.

Merge Sort

Construcción visual paso a paso.

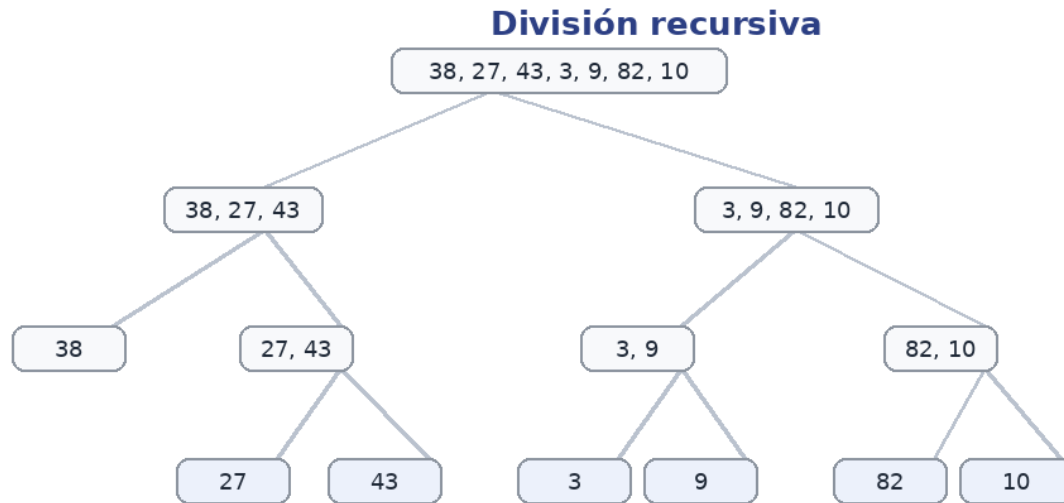


Paso 2

Se separa el arreglo en dos mitades de tamaño similar.

Merge Sort

Construcción visual paso a paso.



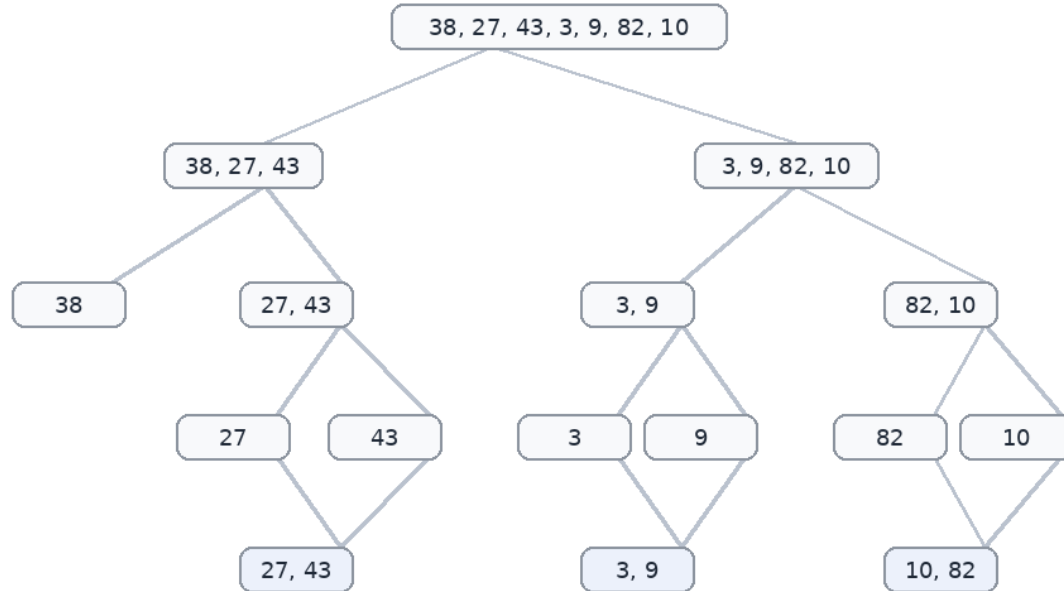
Paso 3

La división continúa recursivamente hasta aislar subproblemas pequeños.

Merge Sort

Construcción visual paso a paso.

Casos base y primeras combinaciones

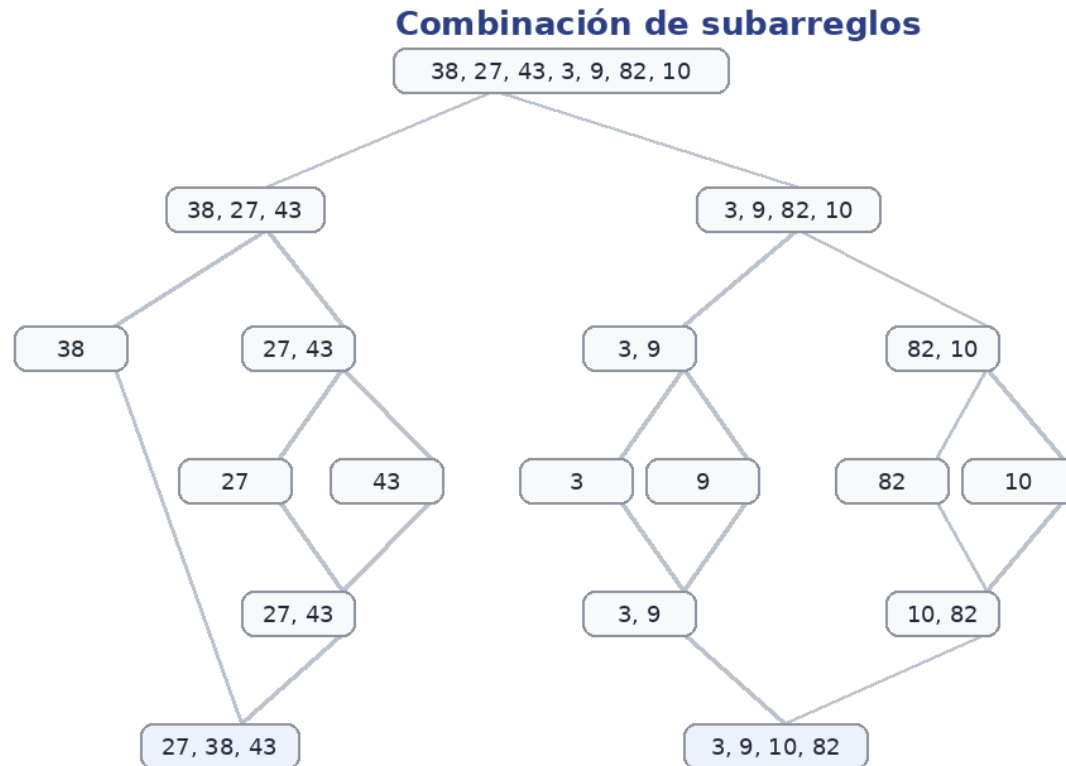


Paso 4

Cada subarreglo de tamaño 1 ya está ordenado: es el caso base.

Merge Sort

Construcción visual paso a paso.

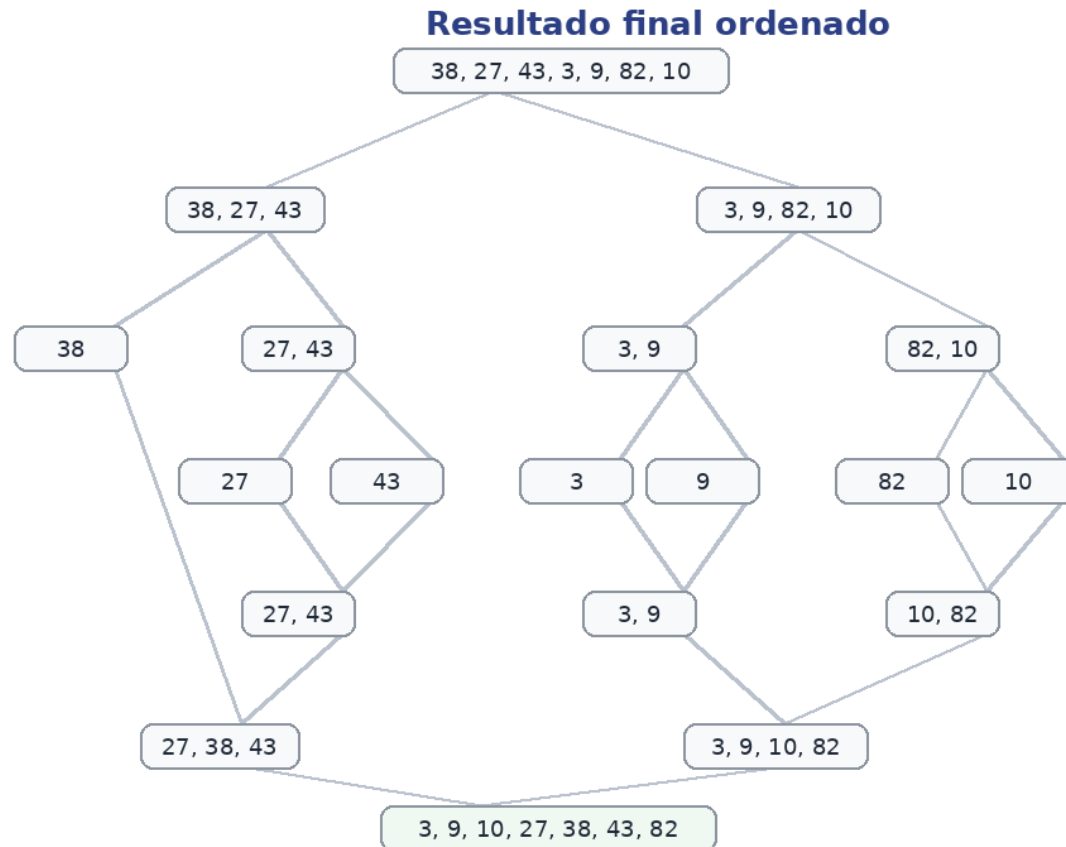


Paso 5

Ahora se van recomblando soluciones parciales mediante merge.

Merge Sort

Construcción visual paso a paso.

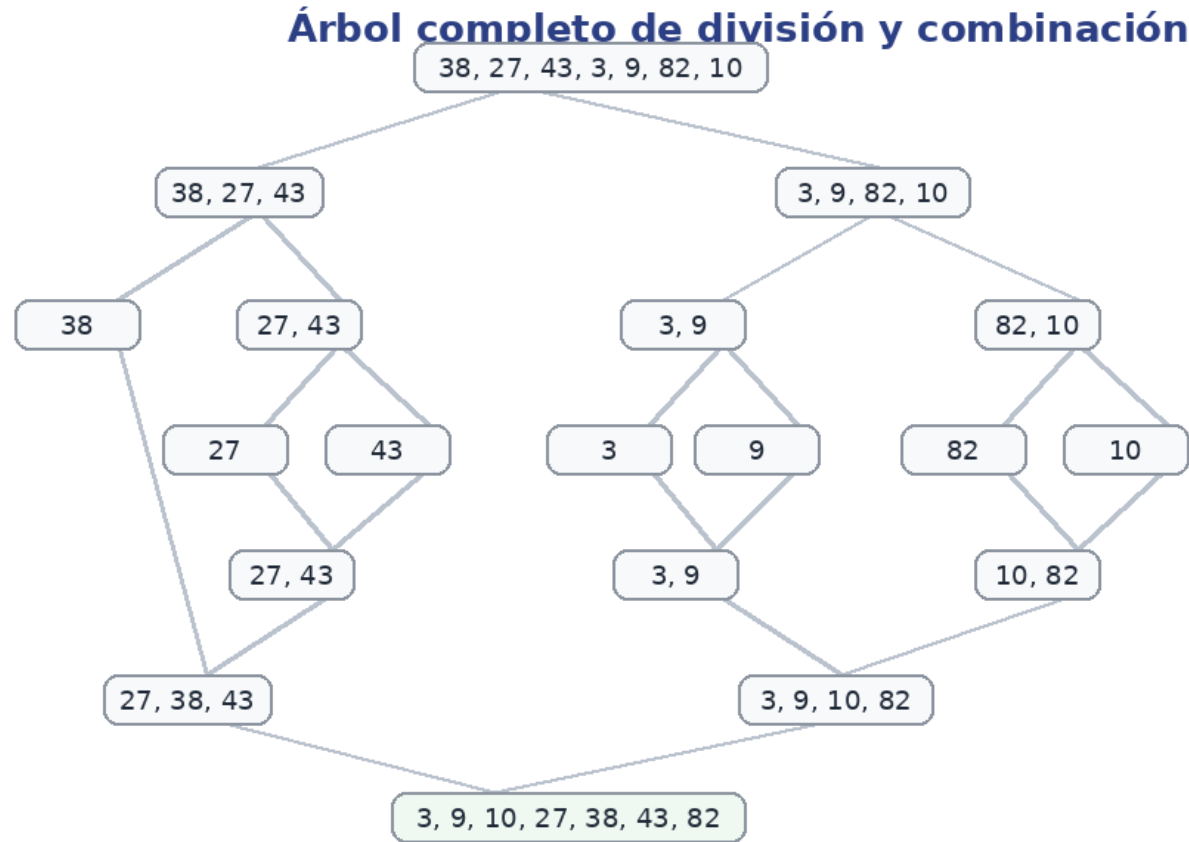


Paso 6

La respuesta final es el arreglo ordenado completo.

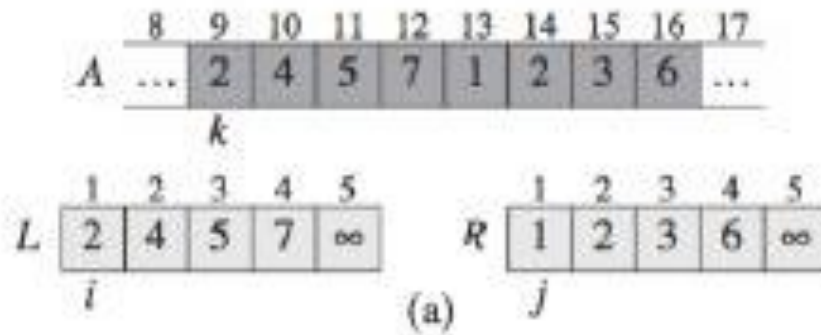
Merge Sort

Árbol completo de división y combinación.



Merge

Invariante del proceso de mezcla.



Invariante

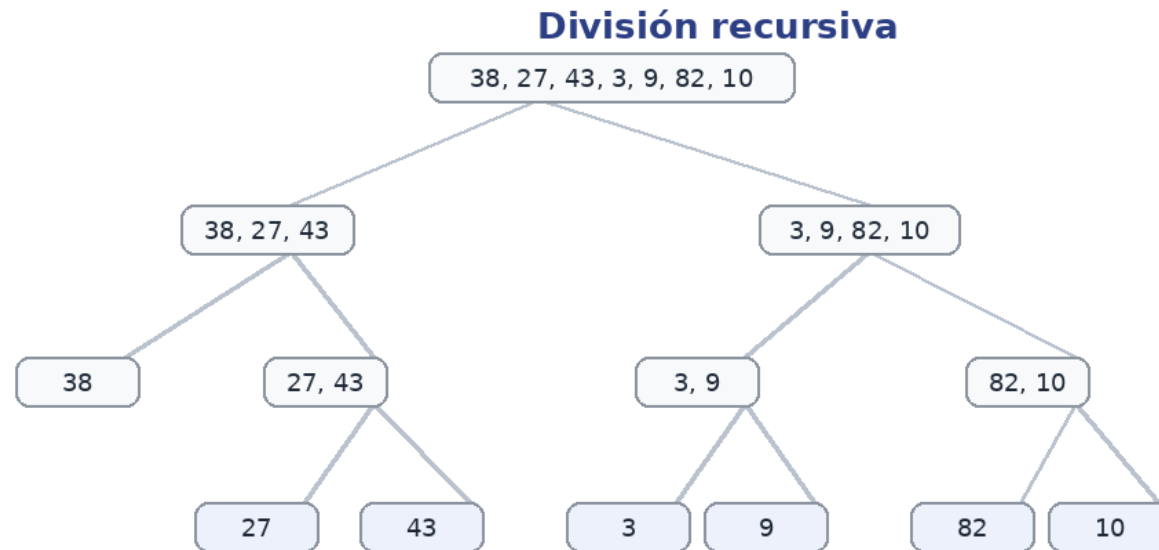
$A[p..k-1]$ contiene los más pequeños.

El resultado parcial está ordenado.

$L[i]$ y $R[j]$ son los candidatos actuales.

Merge Sort

Construcción visual paso a paso.



Descomposición

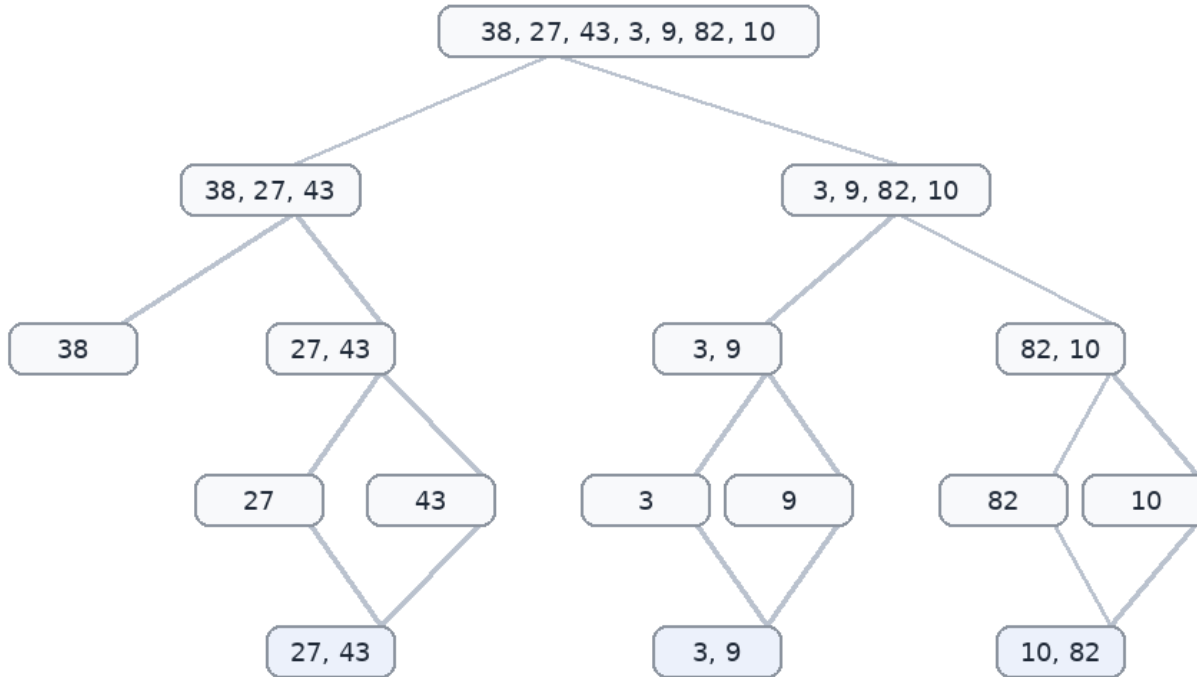
Observar la descomposición.

Cada nivel reparte el trabajo en subproblemas más pequeños.

Merge Sort

Construcción visual paso a paso.

Casos base y primeras combinaciones



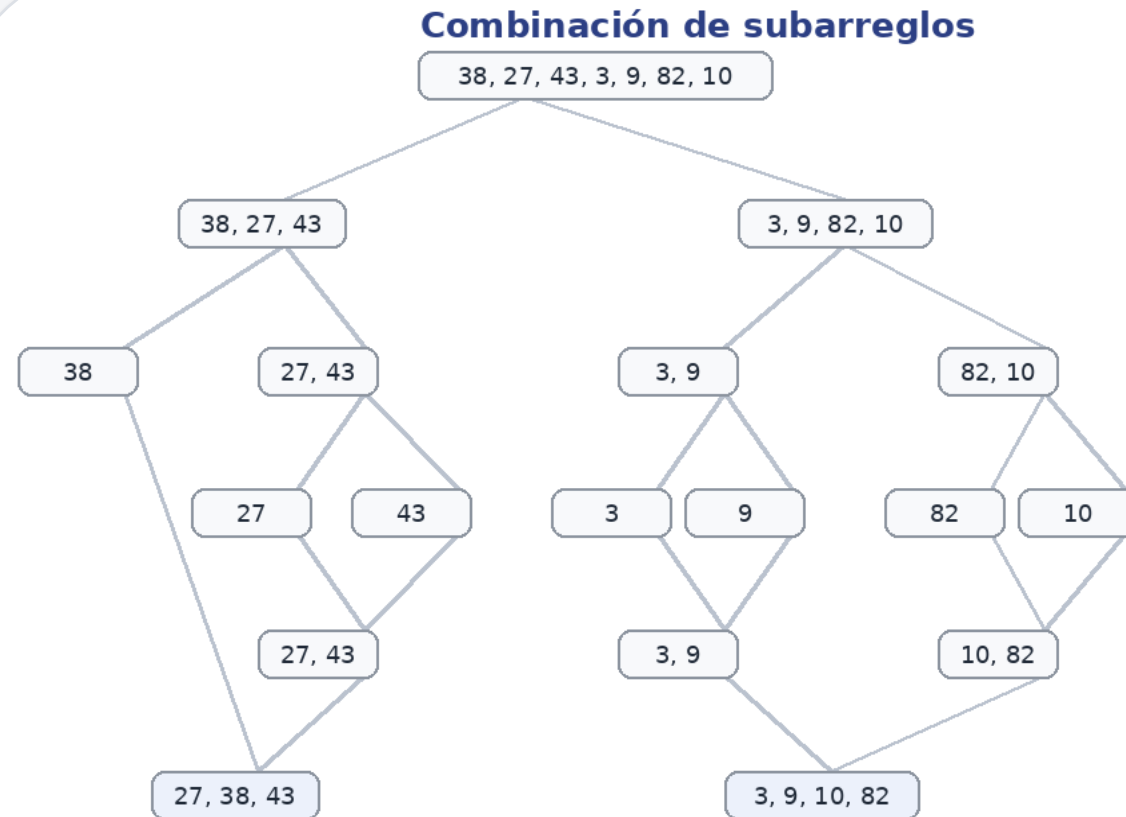
Caso base

Seguir el caso base.

Cuando el tamaño del arreglo es 1, no hace falta ordenar más.

Merge Sort

Construcción visual paso a paso.



Combinación

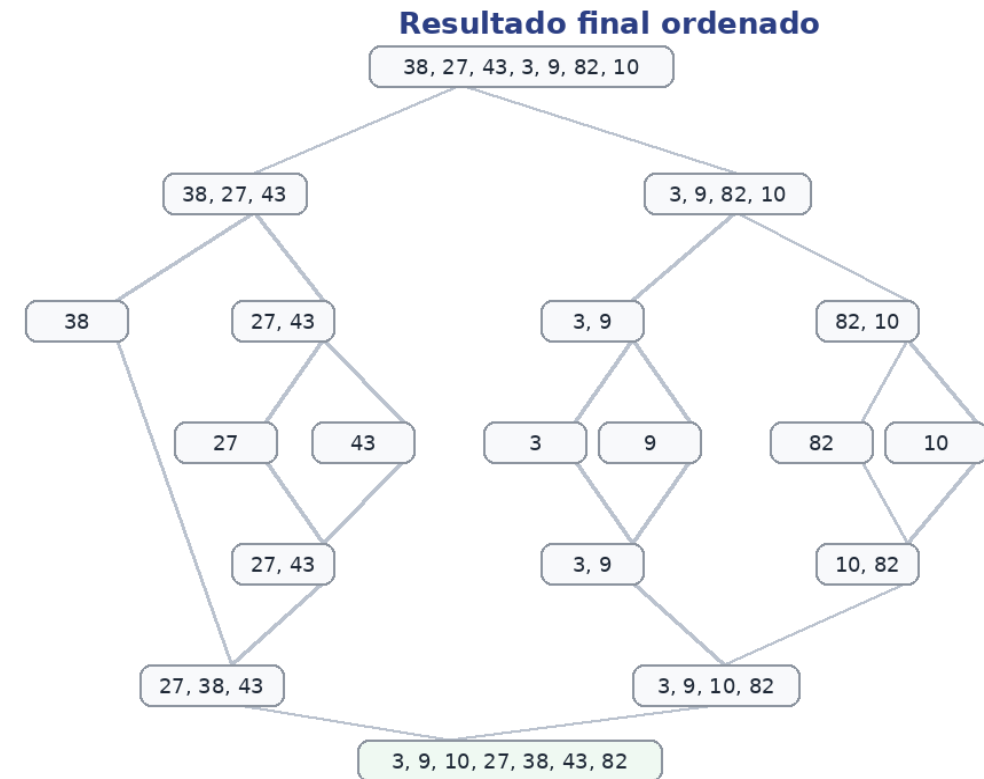
Entender cómo se combina la solución.

La fase merge une subarreglos ya ordenados para construir la respuesta.

Pseudocódigo

Estructura recursiva de Merge Sort.

```
MERGE-SORT(A, p, r)
1  if p < r
2      q = floor((p + r) / 2)
3      MERGE-SORT(A, p, q)
4      MERGE-SORT(A, q + 1, r)
5      MERGE(A, p, q, r)
```



La recursión divide; la llamada a MERGE combina.

Merge

Código e invariante.

```
MERGE (A, p, q, r)
1  copiar A[p..q] en L
2  copiar A[q+1..r] en R
3  i = 0; j = 0
4  for k = p to r
5      if L[i] <= R[j]
6          A[k] = L[i]; i = i + 1
7      else
8          A[k] = R[j]; j = j + 1
```

Invariante

$A[p..k-1]$ contiene los elementos más pequeños ya ordenados.

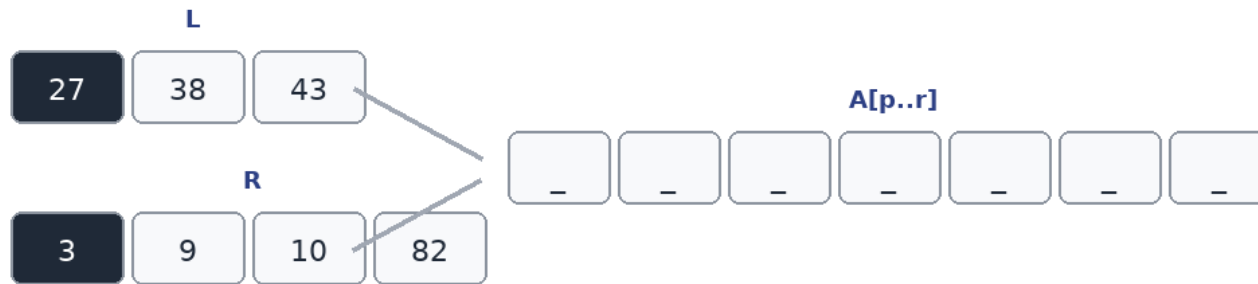
$L[i]$ y $R[j]$ representan los candidatos a entrar.

Cada iteración extiende la respuesta sin romper el orden.

Merge Sort

Construcción visual paso a paso.

Inicio de la mezcla



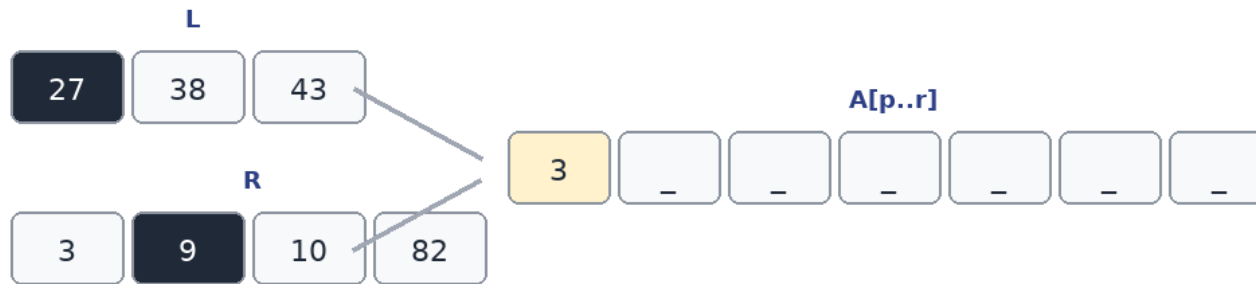
Mezcla 1

Comparamos los primeros candidatos de L y R.

Merge Sort

Construcción visual paso a paso.

Se toma 3 desde R



Mezcla 2

Como 3 es menor, entra primero en el arreglo resultado.

Merge Sort

Construcción visual paso a paso.

Se toma 9 desde R



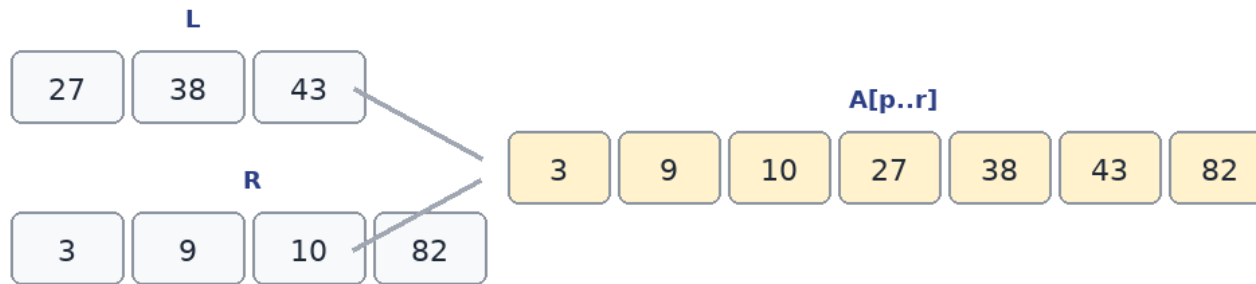
Mezcla 3

Luego 9 también es menor que 27 y se ubica a continuación.

Merge Sort

Construcción visual paso a paso.

Resultado final ordenado



Mezcla 4

Al terminar las comparaciones, la salida queda completamente ordenada.

Análisis

Recurrencia de Merge Sort.

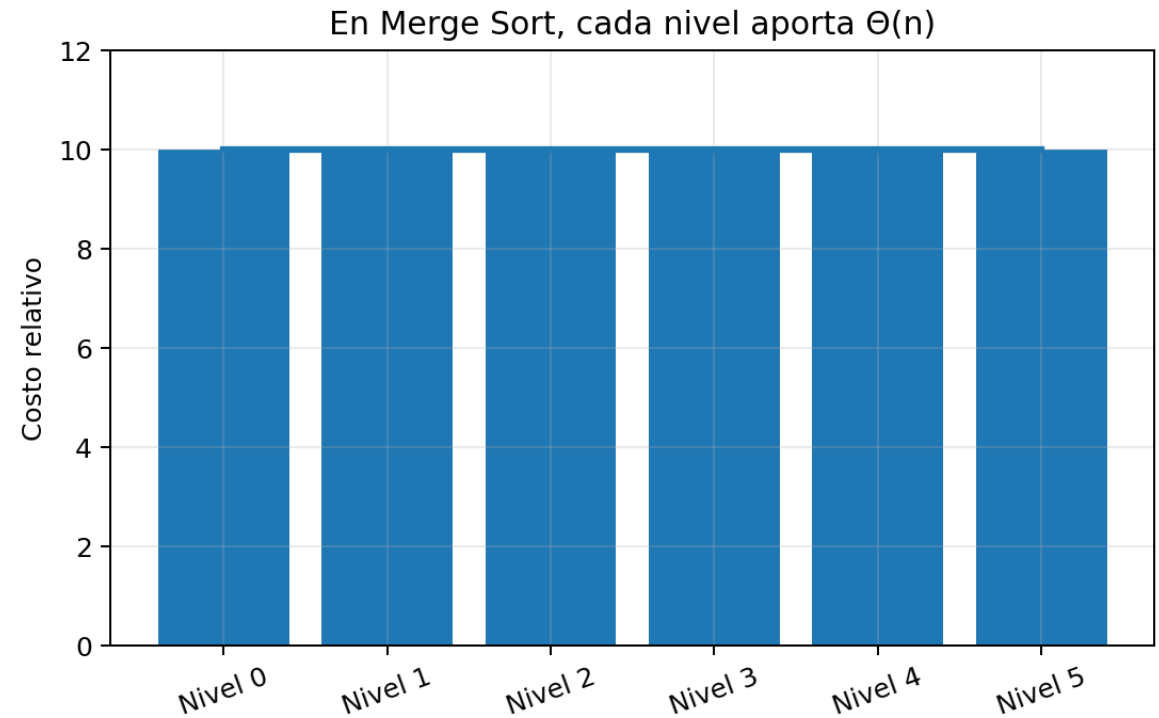
Recurrencia

$$T(n) = 2T(n/2) + \Theta(n)$$

Dos subproblemas de tamaño $n/2$.

Mezclar cuesta tiempo lineal.

La solución es $\Theta(n \log n)$.



25

BLOQUE 2

Teorema Maestro

Una herramienta práctica para resolver recurrencias de divide y conquista.

Teorema Maestro

Receta para resolver recurrencias.

Forma estándar

$$T(n) = aT(n/b) + f(n)$$

a: número de subproblemas

b: factor de reducción del tamaño

f(n): trabajo fuera de la recursión

a

cantidad de
subproblemas

b

factor de
división

f(n)

trabajo de
combinar /
particionar

Tres casos

Comparar $f(n)$ con $n^{\log_b a}$.

Caso 1

$f(n)$ es asintóticamente menor
que $n^{\log_b a}$

$$T(n) = \Theta(n^{\log_b a})$$

Caso 2

$$f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$$

$$T(n) = \Theta(n^{\log_b a} \cdot \log^{(k+1)} n)$$

Caso 3

$f(n)$ es asintóticamente mayor
que $n^{\log_b a}$

$$T(n) = \Theta(f(n))$$

Teorema Maestro

Construcción visual paso a paso.

Comparación guía

Calcular $n^{\log_b a}$.

Comparar $f(n)$ con ese término.

Aplicar el caso correspondiente.

Verificar la condición de regularidad en el caso 3.

Caso 1

$f(n)$ es asintóticamente menor
que $n^{\log_b a}$

$$T(n) = \Theta(n^{\log_b a})$$

Caso 2

$f(n) = \Theta(n^{\log_b a} \cdot \log^k n)$

$$T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$$

Caso 3

$f(n)$ es asintóticamente mayor
que $n^{\log_b a}$

$$T(n) = \Theta(f(n))$$

Aplicación

Complejidad de Merge Sort.

1 $T(n) = 2T(n/2) + \Theta(n)$

2 $a=2, b=2, f(n) = \Theta(n)$

3 $n^{\{\log_b a\}} = n^{\{\log_2 2\}} = n$

4 $f(n) = \Theta(n^{\{\log_b a\}})$

5 Corresponde al caso
2

6 $T(n) = \Theta(n \log n)$

Búsqueda

Comparar la búsqueda lineal con la binaria usando invariantes y complejidad.

Búsqueda

Especificación del problema.

Entrada Arreglo A y valor objetivo x.

Salida Índice i tal que $A[i]=x$, o NIL si no existe.

Observación

Si el arreglo está ordenado, podemos explotar esa estructura para diseñar un algoritmo mucho más eficiente.

Ejemplo: $x = 18$

0	1	2	3	4	5	6	7
2	5	7	9	12	18	21	25

Búsqueda secuencial

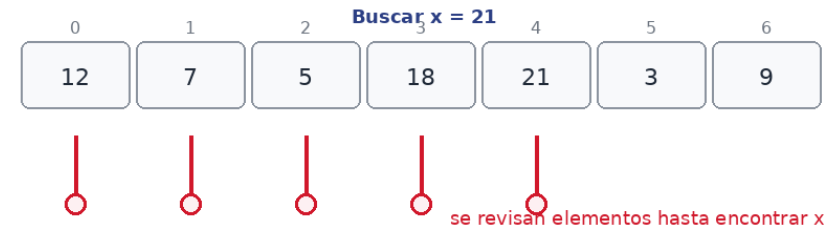
Recorrer hasta encontrar.

Inspecciona cada elemento en orden.

Se detiene si encuentra x.

Si llega al final, concluye que x no está.

Peor caso: $\Theta(n)$.



Código

Búsqueda lineal en Python.

```
def linear_search(A, x):  
    for i in range(len(A)):  
        if A[i] == x:  
            return i  
    return None
```

Idea clave

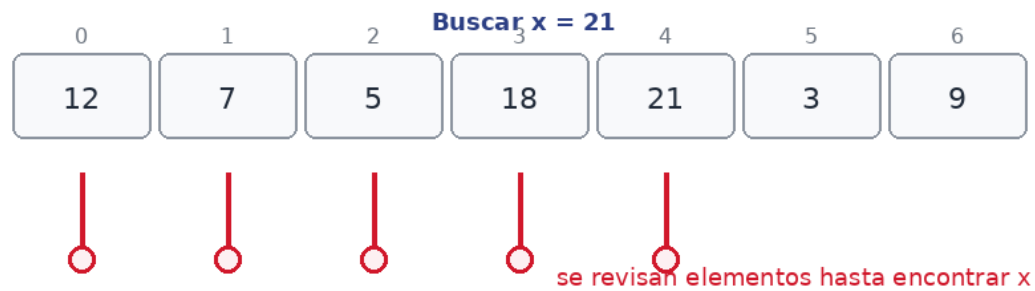
Simple de implementar.

No aprovecha orden en los datos.

Útil cuando el arreglo es pequeño o no está ordenado.

Búsqueda

Construcción visual paso a paso.



Idea clave

Se compara elemento por elemento.

El invariante: los elementos anteriores ya se descartaron.

El costo crece linealmente con n.

Búsqueda binaria

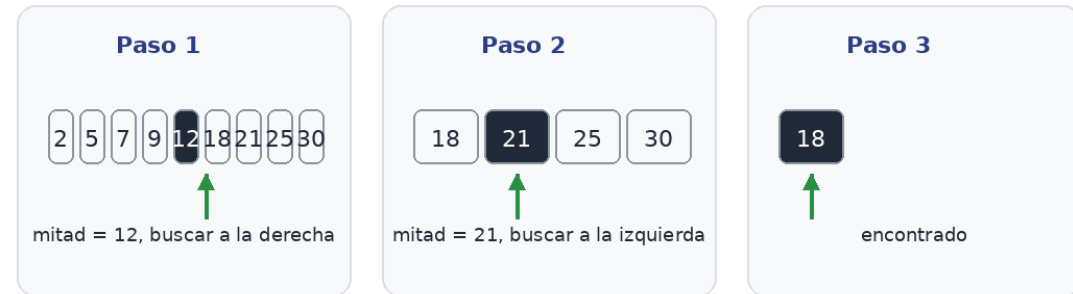
Reducir el espacio de búsqueda.

Requiere que los datos estén ordenados.

Compara con el elemento de la mitad.

Descarta la mitad imposible.

Peor caso: $\Theta(\log n)$.



Código

Búsqueda binaria descendente.

```
def binary_search_desc(A, x):  
    lo, hi = 0, len(A)-1  
    while lo <= hi:  
        mid = (lo + hi) // 2  
        if A[mid] == x:  
            return mid  
        elif A[mid] < x:    # arreglo des  
            hi = mid - 1  
        else:  
            lo = mid + 1  
    return None
```

Observación

La condición del if depende del orden del arreglo.

El intervalo [lo, hi] conserva el objetivo si aún no se descarta.

Búsqueda

Construcción visual paso a paso.

Paso 1

2 5 7 9 12 18 21 25 30



mitad = 12, buscar a la derecha

Paso 2

18 21 25 30



mitad = 21, buscar a la izquierda

Paso 3

18



encontrado

Idea clave

En cada paso, el problema se reduce a la mitad.

El invariante se expresa sobre el intervalo candidato.

Esa reducción explica el logaritmo en la complejidad.

Subarreglo máximo

Transformar un problema de inversión en un problema de suma máxima.

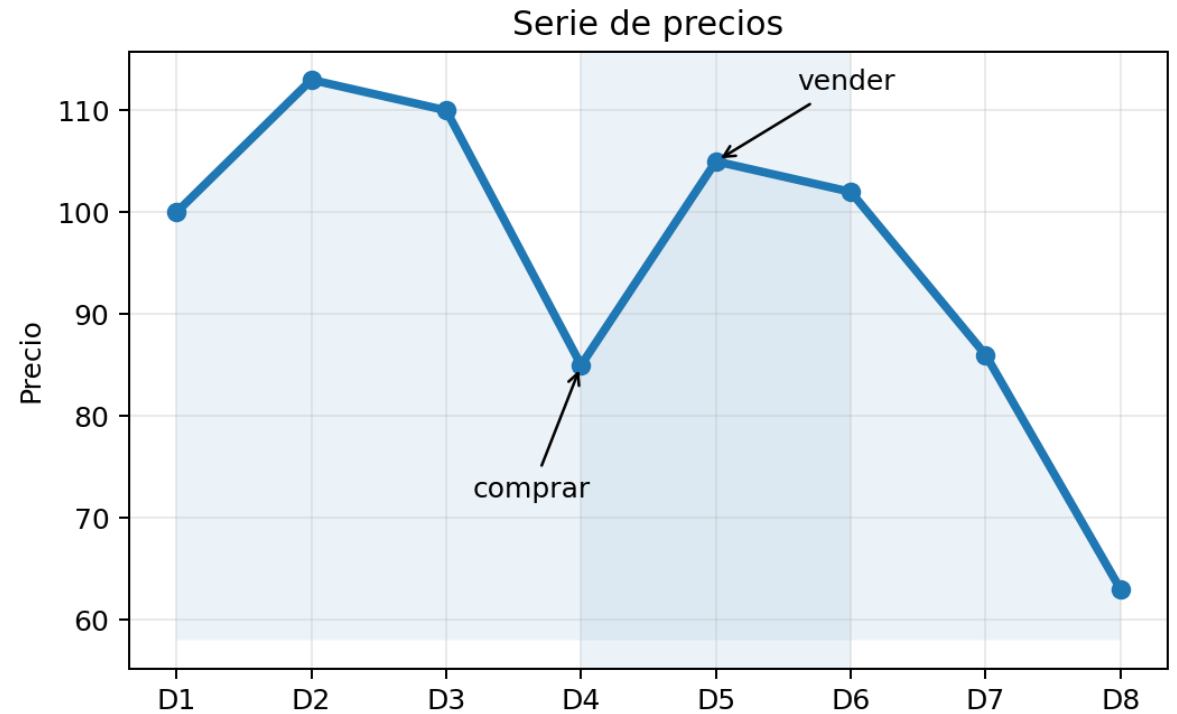
Problema

Maximizar ganancia en precios.

Dado un arreglo de precios por día, decidir cuándo comprar y vender.

La ganancia es $\text{precio}[\text{vender}] - \text{precio}[\text{comprar}]$.

Queremos la mayor ganancia posible respetando el orden temporal.



Estrategia

Primeras ideas para maximizar ganancia.

Idea 1

Probar todas las parejas
(comprar, vender).

Idea 2

Transformar a cambios
diarios.

Idea 3

Aplicar divide y
conquista.

Fuerza bruta

Evaluar todas las parejas posibles.

Idea

Para cada día i , probar cada día $j > i$ como venta.

Calcular la ganancia y conservar la mejor.

Costo: $\Theta(n^2)$.

Matriz conceptual de ganancias

	D1	D2	D3	D4	D5	D6
D1		13	10	-15	5	2
D2			-3	-28	-8	-11
D3				-25	-5	-8
D4					20	17
D5						-3
D6						

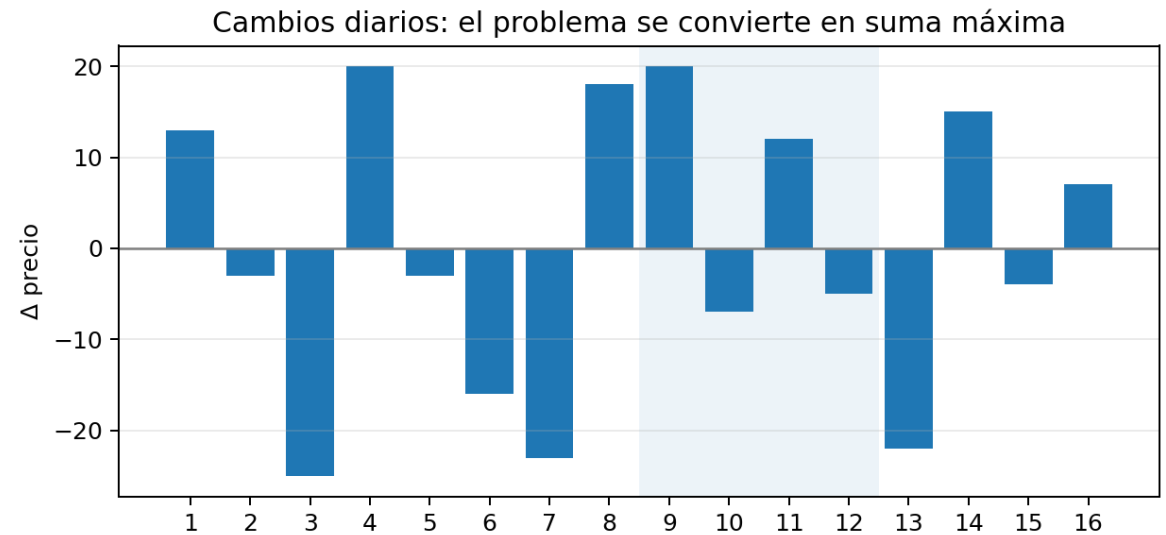
Cambio diario

Del precio al subarreglo máximo.

En lugar de trabajar con precios, calculamos diferencias entre días consecutivos.

La mejor secuencia de compra/venta corresponde al subarreglo de suma máxima.

Ahora el problema se vuelve algorítmico y estructurado.



Divide y conquista

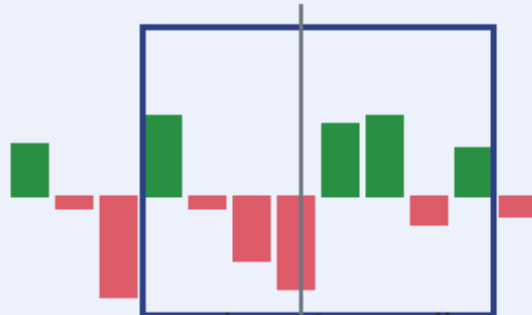
Tres casos del subarreglo máximo.

Izquierda



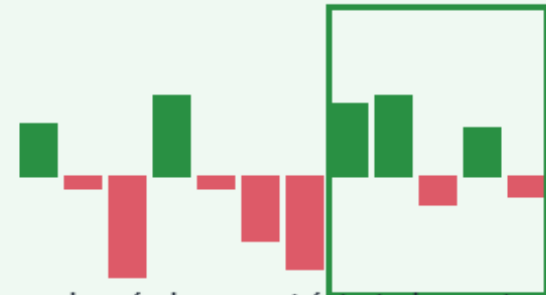
el máximo está totalmente
a la izquierda

Cruzado



cruza el punto medio

Derecha



el máximo está totalmente
a la derecha

Código

Máximo subarreglo cruzado.

```
FIND-MAX-CROSSING(A, low, mid, high)
1  left_sum =  $-\infty$ 
2  sum = 0
3  for i = mid downto low
4      sum = sum + A[i]
5      if sum > left_sum: guardar
6  right_sum =  $-\infty$ 
7  sum = 0
8  for j = mid+1 to high
9      sum = sum + A[j]
10     if sum > right_sum: guardar
11 return left + right
```

Qué hace

Busca la mejor suma terminando en el centro hacia la izquierda.

Busca la mejor suma comenzando en el centro hacia la derecha.

La combinación de ambas forma el mejor subarreglo cruzado.

Código

Máximo subarreglo recursivo.

```
FIND-MAX-SUBARRAY(A, low, high)
1  if high == low
2      return (low, high, A[low])
3  mid = floor((low + high) / 2)
4  left = FIND-MAX-SUBARRAY(A, low, mid)
5  right = FIND-MAX-SUBARRAY(A, mid+1, high)
6  cross = FIND-MAX-CROSSING(A, low, mid, high)
7  return el de mayor suma entre left, right y cross
```

Estructura

Caso base: un solo elemento.

Resolver lado izquierdo y derecho.

Comparar los tres casos: izquierda, derecha y cruzado.

46

CIERRE

Ejercicios

Aplicar diseño D&C, análisis y razonamiento algorítmico.

Ejercicios

Práctica de divide y conquista.

- 1 Diseñar una versión de Merge Sort que ordene de forma descendente y explicar qué cambia en MERGE.
- 2 Resolver las recurrencias $T(n)=3T(n/2)+n$, $T(n)=2T(n/4)+\sqrt{n}$ y $T(n)=8T(n/2)+n^2$ usando Teorema Maestro.
- 3 Comparar experimentalmente búsqueda lineal y binaria para tamaños crecientes de n .
- 4 Aplicar la idea de subarreglo máximo a una nueva serie de cambios diarios e identificar el intervalo óptimo.